1. **Aim of the practical:** Create attendance module for students and teachers.

2. **Objective:** To design and implement an attendance tracking system where:
- Teachers can mark attendance.
- Students can view their own attendance.
- The data is securely stored and associated with course/module info.

3. **Algorithm/Flowchart:**

A) Login.js :

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import API from "../api";
import "./Auth.css"; // add this for UI styling

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [busy, setBusy] = useState(false);
  const navigate = useNavigate();

  const submit = async (e) => {
    e.preventDefault();
    setBusy(true);
    try {
      const { data } = await API.post("/auth/login", { email, password });
      localStorage.setItem("token", data.token);
      localStorage.setItem("user", JSON.stringify(data.user));
      navigate(data.user.role === "teacher" ? "/teacher" : "/student");
    } catch (err) {
      alert(err?.response?.data?.message || "Login failed");
    } finally {
      setBusy(false);
```

B) Login.js :

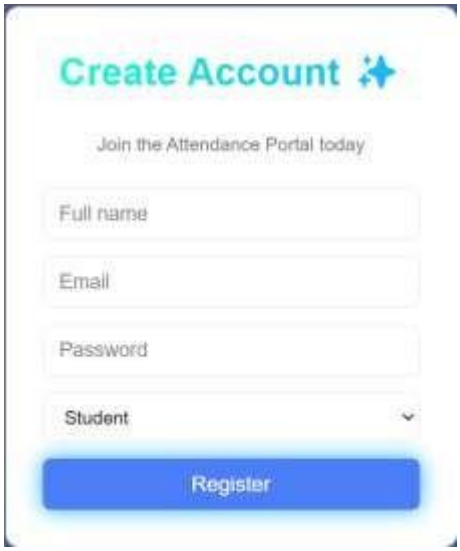
```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import API from "../api";
import "./Auth.css"; // same CSS as login

export default function Register() {
  const [form, setForm] = useState({ name: "", email: "", password: "", role: "student" });
  const [busy, setBusy] = useState(false);
  const navigate = useNavigate();

  const submit = async (e) => {
    e.preventDefault();
    setBusy(true);
    try {
      await API.post("/auth/register", form);
      alert("Registered successfully! Please login.");
      navigate("/login");
    } catch (err) {
      alert(err?.response?.data?.message || "Registration failed");
    } finally {
      setBusy(false);
    }
};
```

C) Server.js:

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');

const authRoutes = require('./routes/authRoutes');
const courseRoutes = require('./routes/courseRoutes');
const attendanceRoutes = require('./routes/attendanceRoutes');

const app = express();
app.use(cors());
app.use(express.json());

// 🟩 ROUTES
app.use('/api/auth', authRoutes);
app.use('/api/courses', courseRoutes);
// app.use('/api/attendance', attendanceRoutes);
app.use("/api/attendance", require("./routes/attendanceRoutes"));


const PORT = process.env.PORT || 5000;
mongoose.connect(process.env.MONGO_URI, { useNewUrlParser: true, useUnifiedTopology: true })
  .then(() => {
    console.log('🟩MongoDB Connected');
    app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
  })
  .catch(err => console.error(err));
```

```
app.get("/seed", async (req, res) => {
const User = require("./models/User");
const Course = require("./models/Course");

await newCourse.save();

res.send("Sample course created!");
    });});
```

4. Result/Output:



Learning outcomes (What I have learnt):

- Learned to design and build a complete web application using the MERN stack (MongoDB, Express.js, React.js, Node.js).
- Implemented **secure login and registration** using JWT (JSON Web Tokens) and bcrypt for password encryption.
- Gained hands-on experience with **role-based access control** (Teacher vs. Student).